VARDHAMAN
COLLEGE OF ENGINEERING

An Internship Project Report on

Route Optimization System Using Dijkstra's Algorithm

ePBL Internship Program 2026

Domain: Data Structures and Algorithms (DSA)

Submitted by

R. Mohananjali	23881A0450
B.Sruthi	23881A0468
S.Anusha	23881A04B8

Supervisor

Mr.S.Srinivas
Associate Professor

Department of Electronics and Communication Engineering

Vardhaman College of Engineering, Hyderabad

May 2026

Contents

Abstract

The Route Optimization System using Dijkstra's Algorithm is a Java-based application developed to determine the shortest path between two cities in a road network. The system represents cities as vertices and roads as weighted edges in a graph. By applying Dijkstra's shortest path algorithm, it calculates the minimum travel distance and displays the optimal route between the selected source and destination.

The project was implemented using object-oriented programming concepts in Java with separate classes for graph representation, edge management, shortest path computation, and user interaction. The application allows users to enter source and destination cities through a menu-driven interface, validates user inputs, and displays the shortest route along with the total travel distance.

This project demonstrates the practical application of graph data structures and shortest path algorithms in solving real-world navigation problems. It also enhances understanding of Java programming, algorithm design, and efficient data processing. The developed system is simple, scalable, and can be further extended by integrating graphical user interfaces, real-time traffic information, and map-based services for advanced route planning.

Keywords: Route Optimization, Dijkstra's Algorithm, Java, Graph Data Structure, Shortest Path, Object-Oriented Programming.

1 Introduction

Route optimization is one of the most important applications of graph algorithms in modern transportation and navigation systems. Finding the shortest path between two locations helps reduce travel distance, time, and fuel consumption. With the increasing need for efficient transportation systems, shortest path algorithms have become essential in logistics, navigation, ride-sharing, and delivery services.

This project presents a Route Optimization System using Dijkstra's Algorithm developed in Java. The system models cities as vertices and roads as weighted edges in a graph. By applying Dijkstra's Algorithm, it computes the shortest route between a user-selected source city and destination city.

The application is implemented using object-oriented programming concepts such as classes, objects, and collections. Users interact with the system through a menu-driven console interface where they can select different operations such as finding the shortest path, displaying available cities, and validating user inputs.

The project demonstrates the practical implementation of graph data structures and shortest path algorithms while providing an efficient and user-friendly solution for route planning. It also serves as a foundation for developing more advanced navigation systems that can integrate real-time traffic information and graphical map interfaces.

2 Literature Review

Route optimization has been widely studied in the field of computer science due to its applications in transportation, logistics, and network routing. Various shortest path algorithms have been proposed, among which Dijkstra's Algorithm remains one of the most efficient methods for graphs with non-negative edge weights.

Several navigation systems, including GPS-based applications, use shortest path algorithms to determine optimal travel routes. Researchers have also explored algorithms such as Bellman-Ford, Floyd-Warshall, and A Search for different routing scenarios. However, Dijkstra's Algorithm is preferred for many practical applications because of its simplicity, accuracy, and efficient performance on weighted graphs.

Recent studies have focused on enhancing route optimization by integrating live traffic updates, road conditions, and artificial intelligence techniques. Although this project uses a static road network, it demonstrates the core principles of shortest path computation and provides a strong foundation for future enhancements involving real-time data and intelligent route planning.

3 Objectives

- To develop a Java-based Route Optimization System using Dijkstra's Algorithm.
- To represent cities and roads using a weighted graph data structure.
- To calculate the shortest path between a source and destination city.
- To provide a menu-driven interface for user interaction.
- To validate user inputs and handle invalid city names gracefully.
- To demonstrate the practical application of graph algorithms in real-world route planning.

4 Methodology

4.1 Block Diagram

4.2 Working Principle

The Route Optimization System operates by representing cities as vertices and roads as weighted edges in a graph. Initially, the user enters the source city and destination city through the console interface. The system validates the entered city names to ensure they exist in the graph.

Once the input is validated, Dijkstra's Algorithm is executed to calculate the shortest path from the source city to the destination city. The algorithm maintains the minimum distance to each city and updates these distances whenever a shorter path is found through neighboring cities.

After all reachable cities are processed, the system reconstructs the shortest path using predecessor information and displays the optimal route along with the total travel distance. If an invalid city is entered or no route exists, an appropriate message is displayed to the user.

4.3 Software Implementation

The project is implemented using Java and follows the principles of object-oriented programming. Multiple classes are used to organize different functionalities of the application.

The **Graph** class manages the road network by storing cities and their corresponding roads using adjacency lists. The **Edge** class represents the connection between two cities and stores the destination city along with the travel distance.

The **Dijkstra** class implements Dijkstra's shortest path algorithm to determine the minimum distance between two cities. The **Main** class provides a menu-driven interface that allows users to enter source and destination cities, validates the inputs, and displays the computed shortest route and total distance.

The application also includes input validation to prevent invalid city names and ensures smooth execution for all supported operations.

5 Results and Discussion

The Route Optimization System was successfully developed and tested using Java. The application correctly implements Dijkstra's Algorithm to determine the shortest path between two cities in a weighted road network. A menu-driven interface was provided to allow users to interact with the system easily.

During testing, the system accepted valid source and destination city names from the user, validated the inputs, and computed the optimal route. The output displayed the sequence of cities forming the shortest path along with the total travel distance. Invalid city names were handled appropriately by displaying an error message without interrupting the execution of the program.

The project was tested with multiple city combinations to verify the correctness of the algorithm. The obtained results matched the expected shortest paths based on the road network, demonstrating the accuracy and efficiency of Dijkstra's Algorithm.

The following figures illustrate the execution of the application:

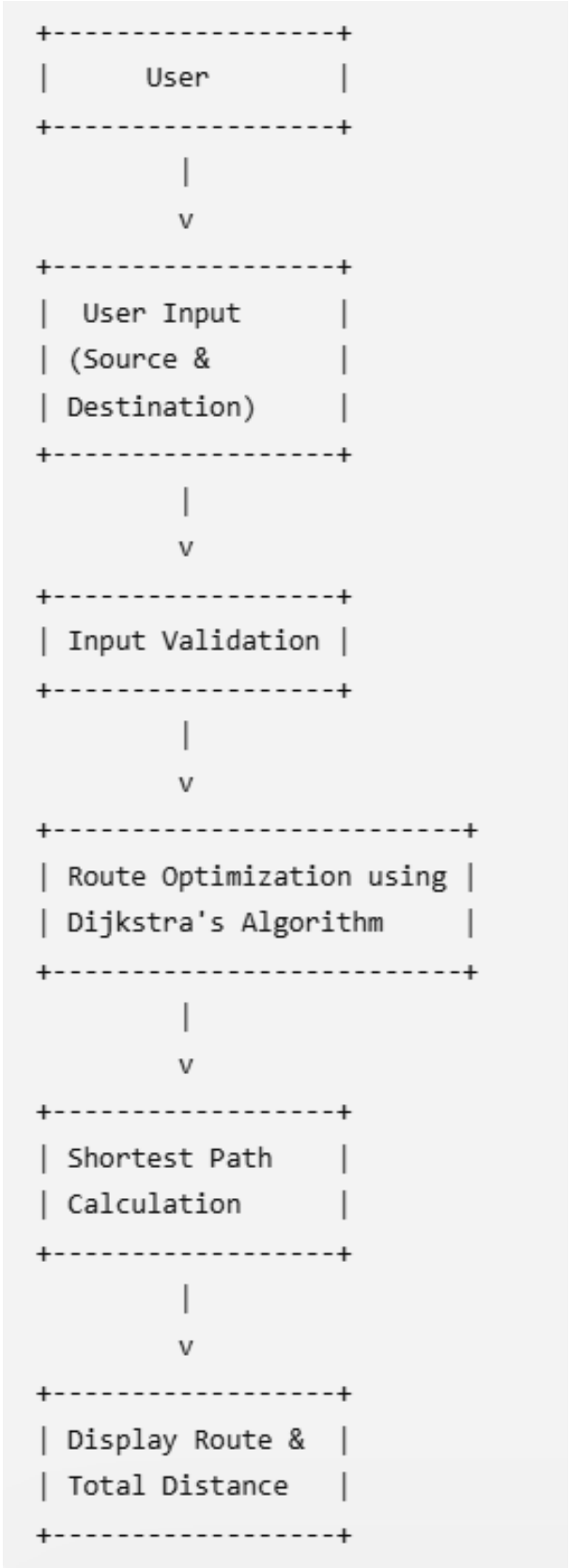

Figure 1: Block Diagram of Route Optimization System using Dijkstra's Algorithm

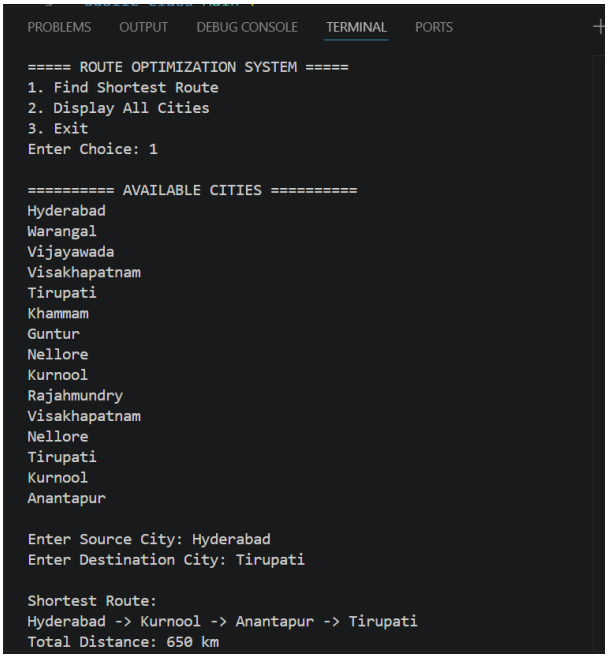

Figure 2: User entering the source and destination cities

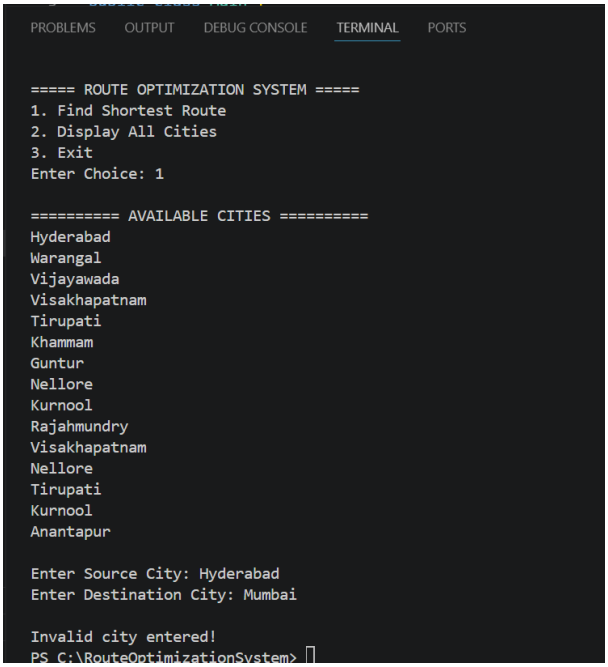

Figure 3: Validation of invalid city names

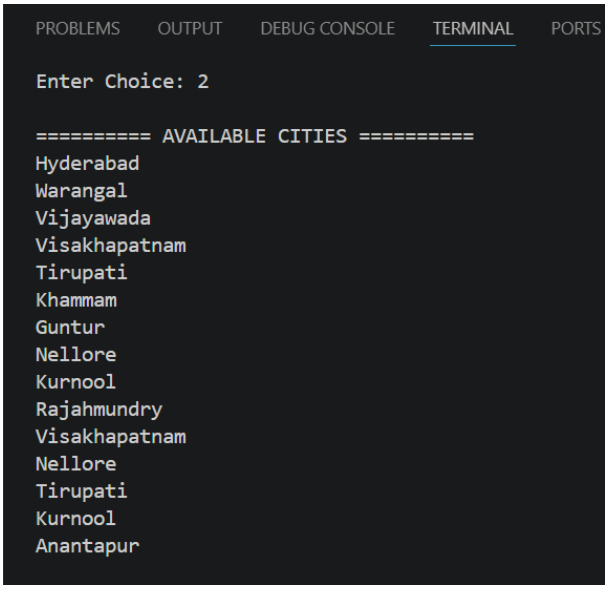

Figure 4: Display of available cities in the network

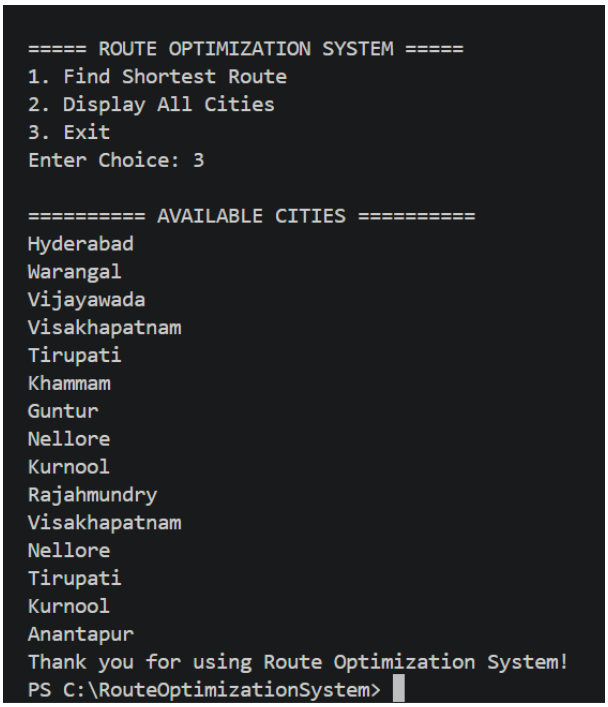

Figure 5: Exiting the page

6 Program Outcomes (POs)

Table 1: Program Outcomes Mapping

PO	**Contribution to the Project**
PO1	Applied engineering knowledge, Java programming, graph data structures, and Dijkstra's Algorithm to develop a Route Optimization System.
PO2	Analyzed the shortest path problem and designed an efficient solution using weighted graphs to determine the optimal travel route.
PO3	Designed and implemented a menu-driven Java application capable of calculating and displaying the shortest route between cities.
PO4	Verified the correctness of the algorithm by testing multiple source and destination combinations and validating the generated routes.
PO5	Used modern software development tools such as Java, Visual Studio Code, Git, GitHub, and Overleaf during project implementation and documentation.
PO12	Enhanced self-learning skills by studying graph algorithms, object-oriented programming, and software development practices while implementing the project.

7 Sustainable Development Goals (SDGs)

Table 2: Sustainable Development Goals (SDGs) Mapping

SDG	**Contribution to the Project**
SDG 9	Industry, Innovation and Infrastructure – The project applies graph algorithms and Java programming to develop an efficient route optimization system, promoting innovation in digital transportation infrastructure.
SDG 11	Sustainable Cities and Communities – The system determines the shortest travel routes, helping improve transportation efficiency, reduce travel time, and support smarter urban mobility.
SDG 13	Climate Action – By identifying shorter routes, the system has the potential to reduce fuel consumption and carbon emissions, contributing to environmentally sustainable transportation.

8 Conclusion

The Route Optimization System successfully demonstrates the implementation of Dijkstra's Algorithm to determine the shortest path between cities using Java. The project models a road network as a weighted graph, where cities are represented as vertices and roads as weighted edges. A menu-driven interface, user input validation, and route display enhance the usability and reliability of the application. The project provided practical experience in graph data structures, shortest path algorithms, object-oriented programming, GitHub version control, and project documentation. In the future, the system can be extended by integrating real-time traffic updates, GPS navigation, interactive maps, and dynamic route optimization to make it suitable for real-world transportation applications.

9 Bibliography

1. E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
   "‘
2. Herbert Schildt, *Java: The Complete Reference*, 12th Edition, McGraw-Hill Education, 2021.
3. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th Edition, MIT Press, 2022.
4. Oracle Corporation, "Java Documentation." Available: https://docs.oracle.com/javase/
5. GeeksforGeeks, "Dijkstra's Shortest Path Algorithm." Available: https://www.geeksforgeeks.org/dijkstra shortest-path-algorithm-greedy-algo-7/
6. GitHub Documentation. Available: https://docs.github.com/
7. Visual Studio Code Documentation. Available: https://code.visualstudio.com/docs

1 Source Code

1.1 Main.java

```
import java.util.Scanner;

public class Main {

    import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Graph g = new Graph();

        g.addCity("Hyderabad");
        g.addCity("Warangal");
        g.addCity("Vijayawada");
        g.addCity("Visakhapatnam");
        g.addCity("Tirupati");
        g.addCity("Khammam");
```

```
        g.addCity("Guntur");
        g.addCity("Nellore");
        g.addCity("Kurnool");
        g.addCity("Rajahmundry");
        g.addCity("Guntur");
        g.addCity("Nellore");
        g.addCity("Kurnool");
        g.addCity("Anantapur");
        g.addCity("Karimnagar");


        g.addRoad("Hyderabad", "Warangal", 150);
        g.addRoad("Hyderabad", "Vijayawada", 275);
        g.addRoad("Warangal", "Vijayawada", 130);
        g.addRoad("Vijayawada", "Visakhapatnam", 350);
        g.addRoad("Vijayawada", "Tirupati", 430);
        g.addRoad("Warangal", "Khammam", 120);
        g.addRoad("Khammam", "Vijayawada", 100);
        g.addRoad("Vijayawada", "Guntur", 35);
        g.addRoad("Guntur", "Nellore", 280);
        g.addRoad("Nellore", "Tirupati", 130);
        g.addRoad("Kurnool", "Hyderabad", 220);
        g.addRoad("Rajahmundry", "Visakhapatnam", 190);
        g.addRoad("Vijayawada", "Rajahmundry", 150);
        g.addRoad("Hyderabad", "Karimnagar", 165);
        g.addRoad("Karimnagar", "Warangal", 80);
        g.addRoad("Vijayawada", "Guntur", 35);
        g.addRoad("Guntur", "Nellore", 280);
        g.addRoad("Nellore", "Tirupati", 135);
        g.addRoad("Kurnool", "Anantapur", 150);
        g.addRoad("Hyderabad", "Kurnool", 220);
        g.addRoad("Anantapur", "Tirupati", 280);
        g.addRoad("Rajahmundry", "Visakhapatnam", 210);
        Scanner sc = new Scanner(System.in);
        int choice;

        System.out.println("\n===== ROUTE OPTIMIZATION SYSTEM =====");
        System.out.println("1. Find Shortest Route");
        System.out.println("2. Display All Cities");
        System.out.println("3. Exit");

        System.out.print("Enter Choice: ");
        choice = sc.nextInt();
        sc.nextLine();

        System.out.println("\n========== AVAILABLE CITIES ==========");

        System.out.println("Hyderabad");
        System.out.println("Warangal");
        System.out.println("Vijayawada");
        System.out.println("Visakhapatnam");
        System.out.println("Tirupati");
        System.out.println("Khammam");
        System.out.println("Guntur");
        System.out.println("Nellore");
        System.out.println("Kurnool");
        System.out.println("Rajahmundry");
        System.out.println("Visakhapatnam");
```

```
77          System.out.println("Nellore");
78          System.out.println("Tirupati");
79          System.out.println("Kurnool");
80          System.out.println("Anantapur");
81
82          if (choice == 1) {
83              System.out.print("\nEnter Source City: ");
84              String source = sc.nextLine();
85
86              System.out.print("Enter Destination City: ");
87              String destination = sc.nextLine();
88              if (!g.cityExists(source) || !g.cityExists(destination)) {
89                  System.out.println("\nInvalid city entered!");
90              } else {
91                  Dijkstra.shortestPath(g, source, destination);
92              }
93
94          } else if (choice == 2) {
95              System.out.println("\nAvailable Cities:");
96              System.out.println("Hyderabad");
97              System.out.println("Warangal");
98              System.out.println("Karimnagar");
99              System.out.println("Khammam");
100             System.out.println("Vijayawada");
101             System.out.println("Guntur");
102             System.out.println("Rajahmundry");
103             System.out.println("Visakhapatnam");
104             System.out.println("Nellore");
105             System.out.println("Tirupati");
106             System.out.println("Kurnool");
107             System.out.println("Anantapur");
108         } else if (choice == 3) {
109             System.out.println("Thank you for using Route Optimization
                    System!");
110         } else {
111             System.out.println("Invalid Choice!");
112         }
113
114         sc.close();
115     }
116 }
117
118 }
```

1.2 Graph.java

```
1  import java.util.*;
2
3  public class Graph {
4
5      private Map<String, List<Edge>> graph;
6
7      public Graph() {
8          graph = new HashMap<>();
9      }
10
11     public void addCity(String city) {
```

```
        graph.putIfAbsent(city, new ArrayList<>());
    }

    public void addRoad(String source, String destination, int distance) {

        graph.get(source).add(new Edge(destination, distance));
        graph.get(destination).add(new Edge(source, distance));
    }

    public Map<String, List<Edge>> getGraph() {
        return graph;
    }
    public boolean cityExists(String city) {
    return graph.containsKey(city);
}
}
```

1.3 Edge.java

```
public class Edge {
    String destination;
    int distance;

    public Edge(String destination, int distance) {
        this.destination = destination;
        this.distance = distance;
    }
}
```

1.4 Dijkstra.java

```
import java.util.*;

public class Dijkstra {

    public static void shortestPath(Graph g, String start, String end) {

        Map<String, List<Edge>> graph = g.getGraph();

        Map<String, Integer> distance = new HashMap<>();
        Map<String, String> previous = new HashMap<>();

        for (String city : graph.keySet()) {
            distance.put(city, Integer.MAX_VALUE);
        }

        distance.put(start, 0);

        PriorityQueue<String> pq =
                new PriorityQueue<>(Comparator.comparingInt(distance::
                    get));

        pq.add(start);
```

```
22
23          while (!pq.isEmpty()) {
24
25              String current = pq.poll();
26
27              for (Edge edge : graph.get(current)) {
28
29                  int newDistance =
30                          distance.get(current) + edge.distance;
31
32                  if (newDistance < distance.get(edge.destination)) {
33
34                      distance.put(edge.destination, newDistance);
35
36                      previous.put(edge.destination, current);
37
38                      pq.add(edge.destination);
39                  }
40              }
41          }
42
43          if (distance.get(end) == Integer.MAX_VALUE) {
44              System.out.println("No route found!");
45              return;
46          }
47
48          List<String> path = new ArrayList<>();
49
50          for (String at = end; at != null; at = previous.get(at)) {
51              path.add(at);
52          }
53
54          Collections.reverse(path);
55
56          System.out.println("\nShortest Route:");
57          System.out.println(String.join(" -> ", path));
58
59          System.out.println("Total Distance: "
60                  + distance.get(end) + " km");
61      }
62  }
```